COMPILER ENGINEERING / RECORDED VALIDATION

A small rewrite, fully inspectable.

IRProof combines a focused LLVM pass, a structured IR generator, translation validation and an evidence viewer. The result is a compact engineering project with results that can be inspected and reproduced.

25	09	01
REFINEMENTS PROVED	REWRITES REFUTED	CHECK LEFT UNKNOWN

The problem is in the contract

A replacement that looks obvious over ordinary arithmetic can fail under LLVM integer semantics. Signed division by two is the smallest example: division rounds toward zero, while an arithmetic right shift rounds down. Overflow flags and undefined values create further restrictions. [1]

sdiv i32 -3, 2 = -1
ashr i32 -3, 1 = -2

The captured validator output for `signed-minus-three` reports the source value as -1 and the target value as -2. This is a real rejected pair, retained in the evidence directory.

What was built

The original code includes a C++20 New Pass Manager plugin, a read-only audit, three IRBuilder specimens, 29 original cases, execution and integrity scripts, a responsive viewer, and an LLVM-23 compatibility patch for Alive2. Four public regression files are retained unchanged under their upstream license.

Recorded environment: LLVM 23.1.4; Alive2 c9af7dbb5038 plus the included patch; Z3 5.3.1. All 35 pairs were executed. There were no tool errors or unexecuted checks.

The contribution is implementation and evidence integration. The project does not claim a new arithmetic identity, a new solver, an upstream-accepted patch or a hardware speed improvement.

01 / COMPILER DESIGN

Focused transformations, explicit preconditions.

Component	Implemented responsibility
Function pass	Scalar integer rewrites, flag handling, definedness checks and branch-sensitive sign information.
Module audit	CFG and SSA edges; instructions and flags; immediate dominators; loop depth and backedges.
C++ frontend	Verified divide, diamond and loop specimens constructed with LLVM IRBuilder.
Evidence runner	Exact source/target execution, conservative verdicts and file provenance.

Transformation rules

Source pattern	Replacement and guard
mul x, 2^k	shl x, k for a positive signed constant; existing nsw/nuw are transferred.
udiv x, 2^k	lshr x, k; preserve exact when present.
urem x, 2^k	and x, (2^k - 1).
sdiv x, 2^k	Identity for 1; exact arithmetic shift when exact; logical shift for known nonnegative values; otherwise a bias expansion.
Neutral operations	Remove add/sub/or/xor by zero and and by all ones.

General signed lowering

```
stable = freeze x  ; omitted when x is known defined
sign   = ashr stable, W - 1
bias   = and sign, (2^k - 1)
q      = ashr (stable + bias), k
```

The freeze is a conservative way to share one operand choice across the expansion. The bias addition receives no invented overflow flags. Branch-derived sign facts are used only for defined operands. The pass does not alter blocks or edges and preserves CFG analyses. [2]

A signed-negative sign-bit multiplier is deliberately skipped. For x = 1, `mul nsw i8 x, -128` is defined, but `shl nsw i8 x, 7` is poison. The captured negative case verifies that distinction.

02 / VALIDATION AND PUBLIC CORPUS

Evidence for each outcome.

Source of cases	Proved	Refuted	Unknown
Original pass and frontend checks	23	0	0
Original deliberately invalid pairs	0	7	0
Original atomic-memory probe	0	0	1
Four unchanged Alive2 regressions	2	2	0
Total	25	9	1

A verdict needs a completed check

The runner accepts a proof only from a complete recognized Alive2 summary with exactly one correct transformation and a successful process exit. It keeps refutations, failed proofs, unsupported semantics, signals, input errors and unexecuted cases separate. Manifest expectations are used to fail tests, never to manufacture the displayed verdict.

The atomicrmw specimen is unknown because the validator reports an unsupported instruction. It is not a proof. Loop IR is included for frontend and graph inspection; it is excluded from the 35 claimed refinement checks.

Two complementary test layers

Compiled execution: 1,690 before/after comparisons run through LLVM lli. These include exhaustive signed 8-bit inputs for six operations, boundary inputs at 16, 32, 64 and 128 bits, defined inputs for exact division, and a guarded sign-bit case. All passed.

Failure handling and interface: 12 Python unit tests cover verdict classification, including partial output and crashes. Four browser arithmetic tests cover the numeric lab. Browser checks exercise search, verdict filters, raw output, provenance, CFG and SSA inspection, JSON export, keyboard tabs and mobile layout.

Public data is a regression corpus

The four public Alive2 files exercise freeze/undef, an overflow intrinsic, and the two directions of poison refinement. Their full upstream revision, source paths, SHA-256 hashes and MIT license are included. This is a focused public test corpus, not a statistical benchmark or a representative sample of all LLVM programs. [3]

Each record retains input, target and raw-output hashes, commands, exit status and timings. Integrity checks establish file consistency; they neither rerun the solver nor authenticate the capture.

03 / REPRODUCIBILITY AND LIMITS

Reproduce with a clear scope.

Use matching LLVM 23 headers and command-line tools, a C++20 compiler, CMake, Ninja and Python. The pinned validator bootstrap additionally needs Git, re2c and pip. Full commands and platform notes are in `docs/reproducibility.md`.

```
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=Release -DLLVM_DIR="$LLVM_DIR"
cmake --build build --parallel 2
ctest --test-dir build --output-on-failure

python scripts/capture_frontend.py
python scripts/make_corpus.py
python scripts/irproof.py run --alive-tv "$ALIVE_TV" --strict
python scripts/verify_evidence.py
python scripts/export_demo.py
```

Compatibility is part of the record

The supplied patch adapts the pinned Alive2 tree to LLVM 23 APIs, removes enum cases that require LLVM 24, adds an archive-build version label, and avoids a second solver runtime through LLVM shared linkage. It leaves the SMT refinement rules unchanged. The exact revision, patch hash and captured binary hash are recorded in the lock file.

Limits that matter

The pass handles scalar integers and a bounded set of local rewrites. It does not perform vector, floating-point, memory or interprocedural optimization. Signed lowering may add instructions; target-specific performance is not measured.

Alive2 checks refinement within its supported model and resource bounds. A successful pair is not a proof of every execution of the entire pass. The loop specimen is used for analysis only. Unknown checks and tool errors must remain visible. [3, 4]

The standalone browser file is a viewer for completed executions. Its BigInt lab evaluates concrete integers and does not model poison, undef or memory. Topology comes from LLVM analysis; graph coordinates come from the renderer.

Primary references

[1] LLVM Project. LLVM Language Reference Manual. Integer operations, flags and freeze.
llvm.org/docs/LangRef.html

[2] LLVM Project. Writing an LLVM Pass (New Pass Manager).
llvm.org/docs/WritingAnLLVMNewPMPass.html

[3] AliveToolkit. Alive2 source and regression corpus, revision c9af7dbb503878554aefafddba5889ccab237503.
github.com/AliveToolkit/alive2

[4] Lopes, N. P., Lee, J., Hur, C.-K., Liu, Z., and Regehr, J. (2021). Alive2: Bounded Translation Validation for LLVM. PLDI 2021.
doi.org/10.1145/3453483.3454030